

AWS Services Guide

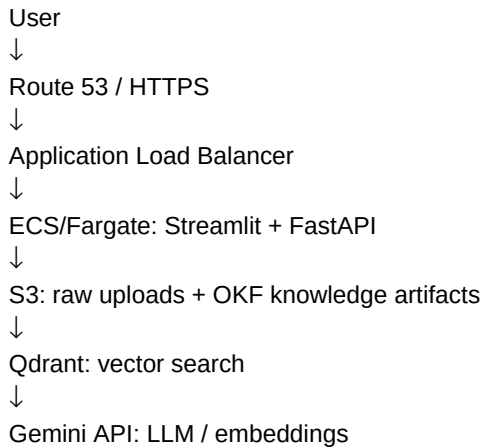
Practical AWS reference for a Dockerized FastAPI + Streamlit + Qdrant + Gemini OKF/RAG application

Scope: What each major AWS service does, why it is useful, and how it can fit this project.

Your current application uses FastAPI, Streamlit, Docker, Qdrant, Gemini, an OKF Markdown knowledge repository, document ingestion, metadata extraction, and retrieval. AWS can provide the cloud infrastructure around these components without requiring you to replace Gemini or Qdrant.

Core recommendation: use ECS/Fargate for containers, S3 for durable documents and OKF artifacts, ECR for images, IAM and Secrets Manager for security, ALB/VPC for networking, and CloudWatch for operations.

1. Recommended AWS Architecture



Supporting services: ECR, IAM, Secrets Manager, VPC, CloudWatch, CloudTrail

For a first cloud deployment, ECS/Fargate is simpler than operating Kubernetes yourself. If Kubernetes is specifically a learning or deployment requirement, EKS is the advanced alternative.

Keep the architectural boundaries clear: S3/knowledge is the durable document layer, Qdrant is the retrieval index, and Gemini is the generation layer.

2. Compute & Containers

Amazon ECS

What it is: Managed container orchestration for Docker workloads.

Why it is used: Your application is already containerized, so ECS is a natural path from Docker Compose to AWS.

How it is used: Build images, push them to ECR, define ECS task definitions, and run ECS services with a desired task count.

Project fit: Run FastAPI and Streamlit as separate services.

AWS Fargate

What it is: Serverless compute for ECS containers where AWS manages the underlying servers.

Why it is used: Avoids EC2 instance administration and is convenient for a small production-like PoC.

How it is used: Specify CPU, memory, image, ports, environment, networking, and IAM task role in the ECS task definition.

Project fit: Run API/UI containers without maintaining worker VMs.

Amazon EKS

What it is: Managed Kubernetes control plane.

Why it is used: Useful if you want Kubernetes-native deployments, Helm, ingress, autoscaling, and deeper DevOps practice.

How it is used: Deploy Kubernetes Deployments/Services/Ingress and connect workloads to AWS resources through IAM integrations.

Project fit: Strong advanced option because the project already uses Docker/Kubernetes concepts, but more complex than Fargate.

Amazon EC2

What it is: Virtual machines in AWS.

Why it is used: Useful when you need OS-level control or want the closest cloud equivalent to a Docker-on-VM setup.

How it is used: Launch an instance, install Docker, configure security groups, and run containers.

Project fit: Good migration path for a simple PoC, but you maintain the VM.

3. Storage & Databases

Amazon S3

What it is: Durable object storage.

Why it is used: Ideal for PDFs, Markdown, TXT, JSON, generated OKF files, backups, and other artifacts.

How it is used: Create prefixes such as uploads/, raw/, knowledge/, and backups/. Access S3 through IAM roles.

Project fit: Recommended durable storage for uploaded source documents and generated OKF Markdown.

Amazon EBS

What it is: Persistent block storage for EC2.

Why it is used: Useful when self-hosting Qdrant or another database on EC2 and needing persistent disk.

How it is used: Attach and mount an EBS volume and keep database data there.

Project fit: Useful for a self-managed Qdrant-on-EC2 architecture.

Amazon RDS

What it is: Managed relational database service.

Why it is used: Useful for structured metadata such as users, ingestion jobs, document records, and audit information.

How it is used: Use PostgreSQL/MySQL privately in a VPC and retrieve credentials from Secrets Manager.

Project fit: Optional; it complements rather than replaces Qdrant.

Amazon DynamoDB

What it is: Managed NoSQL key-value/document database.

Why it is used: Useful for high-scale, simple job state, sessions, or document processing records.

How it is used: Create tables keyed by job/document ID and access them using IAM.

Project fit: Optional; use when the data model favors key-value/document access.

4. Networking & Security

Amazon VPC

What it is: Private networking boundary for AWS resources.

Why it is used: Keeps internal workloads and databases away from direct public exposure.

How it is used: Use public subnets for internet-facing entry points and private subnets for internal services; control traffic with security groups.

Project fit: Foundation for a production-style deployment.

Application Load Balancer

What it is: HTTP/HTTPS Layer-7 load balancer.

Why it is used: Provides a stable endpoint and routes requests to healthy containers.

How it is used: Create listeners, target groups, health checks, and routing rules.

Project fit: Front FastAPI and optionally Streamlit.

Amazon Route 53

What it is: Managed DNS.

Why it is used: Provides a friendly domain instead of an ALB hostname.

How it is used: Create DNS records and alias them to ALB or CloudFront.

Project fit: Use for a public application domain.

AWS IAM

What it is: Identity and access management.

Why it is used: Controls which users and workloads can access S3, ECR, Secrets Manager, CloudWatch, and other resources.

How it is used: Use least-privilege roles for ECS tasks, CI/CD, and administrators.

Project fit: Critical: do not put AWS access keys in source code or images.

AWS Secrets Manager

What it is: Managed secret storage.

Why it is used: Prevents Gemini keys and database credentials from being committed to Git or baked into images.

How it is used: Store secrets and allow only the required ECS task role to retrieve them.

Project fit: Use for GEMINI_API_KEY and future credentials.

5. CI/CD & Container Registry

Amazon ECR

What it is: Private Docker container registry.

Why it is used: Your project already uses Docker images, and ECR is the natural AWS registry.

How it is used: Build, tag, scan, and push API/UI images; deploy specific immutable tags.

Project fit: Store your production container images.

AWS CodeBuild

What it is: Managed build service.

Why it is used: Can run tests, linting, Docker builds, security checks, and image publishing without maintaining a build server.

How it is used: Define build phases in buildspec.yml and authenticate to ECR.

Project fit: AWS-native CI option; GitHub Actions/Jenkins can also do this.

AWS CodePipeline

What it is: CI/CD orchestration service.

Why it is used: Connects source, build, approval, and deployment stages.

How it is used: Typical flow: source → CodeBuild → ECR → ECS deployment.

Project fit: Useful for demonstrating an end-to-end AWS DevOps pipeline.

AWS CodeDeploy

What it is: Deployment automation.

Why it is used: Helps automate controlled deployments and supported blue/green strategies.

How it is used: Configure deployment groups and deployment hooks for the chosen compute target.

Project fit: Useful when releases need stronger rollout control.

6. Monitoring, Logging & Audit

Amazon CloudWatch

What it is: Monitoring, logs, metrics, dashboards, and alarms.

Why it is used: Your current Docker logs are essential for diagnosing ingestion, Gemini, API, and Qdrant failures. CloudWatch centralizes these in AWS.

How it is used: Send container logs to CloudWatch Logs and create alarms for errors, restarts, CPU, memory, latency, and application metrics.

Project fit: Monitor ingestion failures, API errors, LLM failures, and container health.

AWS CloudTrail

What it is: AWS API activity and audit logging.

Why it is used: Shows who changed AWS resources and supports security investigations.

How it is used: Enable trails and store events in a protected S3 bucket; integrate alerts where appropriate.

Project fit: Important for production governance.

AWS X-Ray

What it is: Distributed tracing.

Why it is used: Helps diagnose latency across API calls and multiple services.

How it is used: Instrument supported services and propagate trace context.

Project fit: Optional for the PoC; valuable as the architecture grows.

7. Web Delivery & AI Options

Amazon CloudFront

What it is: Content delivery network.

Why it is used: Improves edge delivery and can provide an HTTPS edge entry point.

How it is used: Put CloudFront in front of S3 or an HTTP origin such as an ALB and configure caching.

Project fit: Optional for the PoC; useful for public multi-region users.

AWS WAF

What it is: Web application firewall.

Why it is used: Filters malicious or unwanted HTTP traffic and can use AWS managed rules.

How it is used: Attach WAF to CloudFront or ALB and configure rules and rate limits.

Project fit: Recommended once the AI assistant is publicly exposed.

Amazon Bedrock

What it is: Managed AWS platform for foundation-model access.

Why it is used: Provides an AWS-native alternative if you later want to use supported foundation models instead of or alongside Gemini.

How it is used: Call supported models through Bedrock APIs using IAM permissions.

Project fit: Optional; it is an LLM platform alternative, not a requirement.

AWS Lambda

What it is: Event-driven serverless compute.

Why it is used: Useful for lightweight processing triggered by events such as an S3 upload.

How it is used: Create an event trigger and execute a short function.

Project fit: Optional; your heavier ingestion pipeline may be better suited to ECS workers.

8. Service Mapping for Your Project

Requirement	AWS service	Use
Run FastAPI	ECS + Fargate	Managed Docker runtime
Run Streamlit	ECS + Fargate	Managed UI container
Store uploads	S3	Durable source documents
Store OKF Markdown	S3	Durable knowledge artifacts
Run Qdrant	EC2 + EBS / managed option	Persistent vector search
Structured metadata	RDS PostgreSQL	Relational records
HTTPS entry	ALB + ACM	TLS and routing
DNS	Route 53	Application domain
Secrets	Secrets Manager	API keys/credentials
Docker registry	ECR	Private images
Logs/metrics	CloudWatch	Observability
Audit	CloudTrail	AWS activity
CI/CD	CodeBuild + CodePipeline	Build/deploy
Network	VPC	Isolation and routing
Web protection	WAF	HTTP filtering

9. Example Deployment Flow

1. Developer pushes code to Git.
2. CI runs tests and builds Docker images.
3. Images are pushed to ECR.
4. ECS/Fargate deploys API and UI.
5. ALB exposes HTTPS endpoints.
6. User uploads a document.
7. Backend stores the source in S3.
8. Ingestion processes the document.
9. OKF Markdown is written to durable knowledge storage.
10. Concepts are embedded and indexed into Qdrant.
11. User asks a question in Streamlit.
12. FastAPI performs retrieval.
13. Retrieved OKF context is passed to Gemini.
14. Answer and source attribution are returned.
15. CloudWatch records logs and metrics.

For the current PoC, migrate incrementally. First move containers to ECS/Fargate and images to ECR. Then move durable documents to S3. Add Secrets Manager and CloudWatch. Add ALB/HTTPS and DNS when the application is ready for public access.

10. Recommended AWS Roadmap

Phase 1 — Minimum cloud deployment

- ECR for Docker images.
- ECS/Fargate for API and UI.
- S3 for uploads and generated knowledge artifacts.
- Secrets Manager for Gemini credentials.
- CloudWatch for logs.
- IAM least-privilege roles.

Phase 2 — Production networking

- VPC with public/private subnet separation.
- Application Load Balancer.
- ACM for HTTPS certificates.
- Route 53 for DNS.
- Restricted security groups.

Phase 3 — Data reliability

- RDS PostgreSQL if structured metadata is needed.
- Persistent Qdrant storage or a managed vector option.
- S3 versioning and lifecycle policies.
- Backups and disaster recovery.

Phase 4 — DevOps and security

- CI/CD using GitHub Actions or CodeBuild/CodePipeline.
- ECR image scanning and immutable tags.
- CloudTrail audit logging.
- WAF for public endpoints.
- CloudWatch dashboards and alarms.

Phase 5 — Kubernetes path

- Use EKS if Kubernetes is a deployment requirement.
- Deploy with Helm and Kubernetes Services/Ingress.
- Add autoscaling and centralized observability.

Key principle: keep the OKF knowledge repository as the source of truth, Qdrant as the retrieval index, and Gemini as the answer-generation layer. AWS should provide secure, durable, observable infrastructure around these components.